Smaller proofs for K215

and a nine-head axiom emitter

Research contribution for Vladimir Reshetnikov's ProveIt

21 October 2026

Abstract

The accepted 215-state binary Turing machine already has Lean and Rocq correctness endpoints. This report contributes a checked simplification of its seed semantics, a checked universal theorem and literal certificates for drain-loop normalization, and a source-level axiom-emitter transformation with a checked resource-safety bridge. The two modified Lean files shrink from 483 to 363 lines while preserving their public interfaces. A separate Coq development certifies 1,059 drain loops in the inherited 1,426-vertex auxiliary control graph, including exact positive durations and a rank bound of six. The AX9 construction moves preservation of the incoming top into the compiled witness word; its exact and biased forms are distinguished carefully. We compare existing 215-state alternatives and report bounded state-reduction searches. **No verified machine with 214 or fewer states is constructed.** Source reduction, auxiliary-graph reduction, and physical-state reduction are kept separate throughout.

Contents

1 Results, repository baseline, and verification boundaries 3

2 What recent work already simplified 4

3 A checked semantic normal form for the seed bundle 4

3.1 The repeated argument . . . 4

3.2 The equal-name case cannot be discarded . . . 5

3.3 Transport the relation once . . . 6

3.4 Measured size and actual verification . . . 6

4 Drain loops as a reusable arithmetic proof interface 7

4.1 The inherited graph and the new theorem . . . 7

4.2 Ranked composition and its exact scope . . . 8

4.3 What this does and does not remove . . . 8

5 Which existing 215-state alternative is easier to prove? 9

6 AX9: move preservation of the old top into the witness word 10

6.1 The complete arithmetic state . . . 10

6.2 The concrete source edit and a cheaper biased version . . . 11

6.3 Repairing the strong compiler for the biased emitter . . . 11

7 An arity-independent soundness theorem for zero-credit schedules 12

7.1 Why count proofs are only half the interface . . . 12

7.2 Clean zero stacks: finite ghost heads remove the credit premise . . . 13

8 State-reduction experiments and their limits 14

8.1 Context-sensitive merging . . . 14

8.2 The source/physical-state tradeoff . . . 14

8.3 The next concrete synthesis target . . . 15

9 Integration plan and reproduction 15

9.1 Immediate integration: the semantic patch . . . 15

9.2 Drain and AX9 replay . . . 15

9.3 Priorities after integration . . . 16

A Archive contents and trust boundaries 16

1 Results, repository baseline, and verification boundaries

The requested target is a deterministic binary, one-tape Turing machine M for which the intended mathematical formalization establishes

$$\mathrm{ZFC} \vdash (\neg \mathrm{Halt}(M, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{ZFC})).$$

The repository's existing exact K215 machine is the baseline. Its Lean theorem states blank halting if and only if ZFC has a finite syntactic refutation. Rocq has a corresponding exact-table endpoint. These external proof-assistant statements, the ordinary mathematical internalization argument, and a separately formalized internal PA derivation are distinct deliverables; the latter is still open in the repository. Nothing here asserts unconditional nonhalting or a minimum-state theorem. See the pinned verification and candidate records [1, 2].

The primary source review is pinned to `b07002a86b`. Intake and literal-graph binding also use snapshot `56e487ce8f`; full commit identities are retained in `provenance.json` and the references. The latter snapshot includes the automatic-certificate packet. The sole remaining incoming ZIP at that snapshot, `K215_Proof_and_Quotient_Detachment_2026-10-21.zip`, was inspected. Its DomRho8 simplification and quotient-detachment theorem are prior concurrent work, not discoveries of this packet [6].

Contribution	Result	Verification boundary
Seed semantics	One bundle/seed equivalence; a relation-congruence lemma; 120 lines and 3,788 bytes removed across two files	Fresh Lean 4.32.0 build of 26 dependency modules and a typed API client. Full 184-module endpoint closure not rebuilt here.
Drain normalization	Exact $R_r + 1$-step affine drain theorem; all 1,059 literal instances; rank at most six; 367 retained vertices	Coq 8.18 compilation and recursive kernel audit. Auxiliary control graph, not a new binary-machine endpoint.
AX9 and resource safety	Exact AX10 factorization; restricted biased factorization; opaque-tail execution soundness from prefix credit	Standalone Coq 8.18 development, twelve closed assumption reports and recursive kernel audit. Numerical-calculus interfaces are explicit parameters.
Source construction	Clear and biased AX9 sources; deterministic edits; complete retained-edge comparison	Reproducible source artifacts. Source arithmetic arguments and finite checks do not certify a binary table.
State search	Context-sensitive fold/provider searches and protected-row allocation experiments	Bounded negative results only; no 214-state result and no global lower bound.

The accepted native table and the exact edited-source input are unchanged by the Lean patch. Their full SHA-256 identities are recorded in `provenance.json`.

2 What recent work already simplified

A useful proposal must start after the current repository, rather than repeat the first normalization or symmetry observation. The phase-five checkpoint already uses ordinary source bisimulation: all 2,240 offsets were zero, so the source endpoint no longer needs the integer-offset proof. Its generic soundness theorem is parameterized by a path decoder. K215 retains its original long-path decoder; substituting the shorter legacy decoder changes 622 certificate entries and is not a harmless implementation detail [3].

The current Lean closure is 184 modules. The phase-five receipt records 37 freshly compiled affected modules and 147 predecessor objects reused with exact bindings; it is not a claim that every object was freshly recompiled at that checkpoint. Earlier incoming packets also provide the 1,426-entry normalized source checker, symmetry, automatically inferred cross-language correspondence, compact certificate codecs, balanced lookup, shared framed decoder windows, and local marker transport [4, 5]. These are valuable integration work, but they are inherited here.

This report therefore takes two additional steps. First, it removes repeated semantic reasoning about the same quantified axiom bundle. Second, it turns the previous drain-loop research argument into a compiled theorem with literal-instance checks. The latter has value as a reusable arithmetic interface; it is not needed merely to compare two already equal canonical graphs.

The recommended architecture has four reusable interfaces:

1. An exact binary runtime interface, with whole-tape frame conditions, positive progress, and complete site coverage.
2. A source interface whose generated correspondence is checked by a small semantic theorem.
3. A command interface that separates witness compilation, resource safety, and theorem preservation.
4. A logical interface expressed through stable semantic normal forms, rather than repeated decoding of arithmetic syntax.

This organization does not erase obligations. It makes repeated obligations instances of one theorem.

3 A checked semantic normal form for the seed bundle

3.1 The repeated argument

The forward theorem `bundle_of_seed` and reverse theorem `seed_of_bundle`, in the KSeed222 namespace, independently decode the same nested quantified bundle. The forward direction constructs witnesses; the reverse direction recovers them. Both repeat valuation updates, membership conditions, closure conditions, and the fact that the private matrix-code label does not affect the matrix's meaning.

Define, for a binary relation Q and a membership relation $\in$,

$$\begin{aligned}
\mathrm{Request}(Q, A, C) &\iff \forall y \,(Q(A, y) \to \exists z \,(Q(A, z) \wedge z \in C)),\\
\mathrm{Closure}(Q, C) &\iff \forall y \in C \;\exists x \,\Big(Q(x, y) \wedge \forall x' \subseteq y \;\mathrm{Request}(Q, x', C)\Big),\\
\mathrm{Seed}(Q, A) &\iff \exists C \,(\mathrm{Request}(Q, A, C) \wedge \mathrm{Closure}(Q, C)).
\end{aligned}$$

Here $x' \subseteq y$ abbreviates the relation-theoretic subset predicate used by the repository. These are semantic formulas, with no assumption that the underlying membership structure is already a model of all of ZFC.

Let e be a valuation, and let

$$Q(x, y) = \mathrm{Sat}\big(\in, e[a \mapsto x][b \mapsto y], \mathrm{dec}(c)\big).$$

The input name is a, the output name is b, and c is the private label associated with the numerical matrix code.

Theorem 3.1 (Checked bundle normal form)**.** *Assume membership is irreflexive and a, b, c are pairwise distinct. Then*

$$\mathrm{Sat}\big(\in, e, \mathrm{dec}(\mathrm{bundle}_{222}(a, b, c))\big) \quad\iff\quad \mathrm{Seed}(Q, e(a)).$$

This is `KSeed222.BundleSemantics.bundle_iff_seed` in the supplied Lean sources.

Proof structure. Normalize the encoded request and closure separately. The small lemmas `req_iff` and `clos_iff` identify them with the first two formulas above. Valuation updates at distinct labels commute; repeated assignment to one label absorbs the earlier assignment; assigning its current value changes nothing. The inherited `sat_dec_agree` theorem supplies the less syntactic invariant: assigning to the private label c does not change the truth of $\mathrm{dec}(c)$.

The bundle's vacuity factor is true under the distinctness hypotheses. Classical normalization of its negated universal core produces an existential container. The two remaining conjuncts are closure and request, in the opposite order from the definition of Seed. Reordering the conjunction proves the equivalence. The proof invokes neither of the old directional bundle/seed theorems, so there is no circular reuse of the result being refactored. □

The old reverse proof is now a projection:

```
(BundleSemantics.bundle_iff_seed mem hirr e a b c hab hca hcb).mp h
```

3.2 The equal-name case cannot be discarded

The original forward API permits $a = b$. This changes the meaning of nested binders and does not satisfy the new equivalence theorem's hypotheses. A shorter proof obtained by adding $a \neq b$ to the old API would not be an acceptable replacement.

The patch retains the original forward statement and proves the equal-name case separately in `diagonal_bundle_of_seed`. The public proof splits on equality:

```
by
  by_cases hab : a = b
  . subst b
    exact BundleSemantics.diagonal_bundle_of_seed mem hirr e a c hca h
  . exact
      (BundleSemantics.bundle_iff_seed mem hirr e a b c hab hca hcb).mpr h
```

The actual Lean source uses its standard tactic bullets. This separate case is checked, not deferred. The domain and valuation treatment remains the same as the upstream API; no new nonemptiness hypothesis is introduced.

3.3 Transport the relation once

A second reusable lemma states that seedhood depends only on the truth values of its relation:

Theorem 3.2 (Relation congruence)**.** *If $Q(x, y) \leftrightarrow Q'(x, y)$ for every x, y, then*

$$\mathrm{Seed}(Q, A) \leftrightarrow \mathrm{Seed}(Q', A).$$

Its Lean proof is a direct simplification of the definition:

```
theorem seedProp_congr
    (mem : V -> V -> Prop) (A : V)
    {Q Q' : V -> V -> Prop}
    (hQ : forall x y, Q x y <-> Q' x y) :
    SeedProp mem Q A <-> SeedProp mem Q' A := by
  simp only [SeedProp, hQ]
```

The displayed ASCII arrows are for typesetting; the delivered source has the exact Lean syntax. The final witness-by-witness transport in `seed_eqFree` becomes

```
exact (seedProp_congr mem (e 0) hiff).mp hs
```

This is independent of the earlier proposal to eliminate syntactic round-trip inductions using an existing semantic round trip. Those encoding lemmas remain intact. The two changes can therefore be considered separately during integration.

3.4 Measured size and actual verification

Source	Old lines	New lines	Old bytes	New bytes
`KSeed222.lean`	214	195	7,836	8,645
`KSeedModel222.lean`	269	168	11,711	7,114
Total	483	363	19,547	15,759

The combined reduction is 120 physical source lines (24.8%) and 3,788 bytes (19.4%). The first file gains explicit reusable theorem statements; the main byte saving is in the reverse-direction proof. These are source-size measurements, not a benchmark of the full endpoint's build time.

A fresh Lean 4.32.0 replay compiled all 26 modules in this local dependency closure, then the typed audit client. Its four inspected theorem reports contain only

`propext`, `Classical.choice`, `Quot.sound`.

There is no `sorryAx` or added axiom. The public forward, reverse, and equality-free transport signatures are explicitly checked. One inherited unused-section-variable warning on `matrix_iff` remains because its parameter was retained for compatibility; the audit client has no warnings.

This is a checked local replacement. The full downstream K215 endpoint closure must still be rebuilt when integrating it. The packet supplies the exact two-file patch, originals, complete modified local closure, portable replay driver, source bindings, and raw logs.

4 Drain loops as a reusable arithmetic proof interface

4.1 The inherited graph and the new theorem

Prior affine-core research extracted a canonical graph with 1,426 vertices and identified 1,059 drain loops. Its structural and algebraic checks were Python checks; its generic arithmetic and contraction arguments were not compiled [8]. We retain that attribution and input graph. The new `DrainAffine.v` proves the local theorem and ranked normalization in Coq; `DrainInstances.v` checks the literal graph data.

At a vertex v testing register r, suppose the nonzero edge decrements r, adds a nonnegative vector c, and returns to v. The zero edge adds a vector d and exits at w. The essential syntactic condition is $c_r = 0$. The exit increment d_r need not be zero.

Define

$$F(R)_j = \begin{cases} d_r, & j = r, \\ R_j + R_r c_j + d_j, & j \neq r. \end{cases}$$

Theorem 4.1 (Exact drain execution)**.** *For every natural register vector R, with r in range, the described drain reaches $(w, F(R))$ after exactly $R_r + 1$ graph transitions. Consequently*

$$\mathrm{Halt}_G(v, R) \leftrightarrow \mathrm{Halt}_G(w, F(R)).$$

Proof. Let $n = R_r$. If $n = 0$, the zero edge is taken immediately. Its addition of d agrees with F, including the tested coordinate.

For $n + 1$, the first transition decrements the tested coordinate to n because $c_r = 0$. Every other coordinate gains c_j. Apply the induction hypothesis to this state. Its final non-tested coordinate is

$$R_j + c_j + nc_j + d_j = R_j + (n + 1)c_j + d_j.$$

The tested coordinate is d_r. The step count is one plus $n+1$, hence $(n+1)+1$. The exit is therefore a finite positive prefix of the deterministic execution. Halting before and after a deterministic finite prefix is equivalent. □

The implementation uses finite lists for registers. It needs no functional extensionality axiom. Both the arithmetic theorem and halting-prefix lemma are closed under the global Coq context. Control labels use Coq's binary naturals; register values remain arbitrary natural numbers. The first literal

experiment with unary labels exhausted its resource budget, so the successful literal checker uses the same numerical labels in binary representation. This is an implementation choice in the auxiliary proof data, not a change to a Turing machine.

4.2 Ranked composition and its exact scope

Define `normalize fuel G (v,R)` to execute at most `fuel` whole drain macros, stopping when the current vertex is not recognized as a drain. For every finite fuel, the new theorem gives a finite run of the original graph from the input to that normalized configuration and preserves halting. This result does not need a rank: even when drain exits form a cycle, a fixed finite number of macros still expands to a finite run.

Ranks give a stronger stopping statement. If every drain exit strictly lowers a natural rank, and the input rank is at most the fuel, normalization reaches a non-drain vertex. In the literal graph every rank is at most six.

Theorem 4.2 (Literal six-macro normalization)**.** *For every literal graph vertex v and every nine-register vector R, let*

$$(v', R') = \mathrm{normalize}(6, G, (v, R)).$$

Then v' is not a drain, the original graph has a finite execution from (v, R) to (v', R'), and

$$\mathrm{Halt}_G(v, R) \leftrightarrow \mathrm{Halt}_G(v', R').$$

Six bounds the number of drain *macros*, not the number of graph transitions and certainly not the number of binary TM steps. A single drain macro takes $R_r + 1$ graph transitions and can therefore be arbitrarily long.

The literal checks establish:

- 1,426 graph vertices, with unique labels;
- exactly 1,059 drain vertices and complete agreement between the supplied drain list and recognition;
- exact binding of every drain certificate to the literal graph;
- strict exit-rank decrease and global rank bound six;
- all transition targets belong to the literal vertex set;
- exactly one stop vertex and exactly 367 retained vertices;
- the initial vertex 0 is retained.

Target closure matters because the lookup function returns `Stop` for a missing label. A graph with an omitted target must not acquire an unintended halting transition. The explicit finite closure check rules out that input defect.

4.3 What this does and does not remove

The local proof replaces 1,059 repeated loop inductions with one theorem plus literal shape checks. It also supplies a common interface for source programs that implement a transfer, copy, or clear operation with different small loops.

The theorem is currently an equivalence between two configurations *of the same canonical graph*. This packet does not prove a new source-to-canonical-graph adequacy theorem, and it does not claim that a separately encoded 367-node matrix machine has been connected to K215. The previous research's matrix compression remains a separate representation check. No binary state was removed by this proof refactoring.

This distinction suggests an integration order. For the current pair of identical normalized source graphs, the simpler direct normalized-certificate lane remains appropriate. Use drain normalization when it removes an actual low-level implementation mismatch between candidates, or when a compact arithmetic interface simplifies subsequent reasoning. Fewer retained vertices alone do not imply a faster kernel check.

5 Which existing 215-state alternative is easier to prove?

The answer depends on the proof one intends to obtain. For an *additional theorem relative to the already certified K215*, a small local table transport is preferable to repeating the entire source/calculus development. For a standalone explanation of the scheduler, zero-credit enumeration has a different advantage. These comparisons do not assert that an alternative has a smaller transitive proof closure than K215.

Candidate	Advantage	Remaining issue
Primary K215	Existing exact-table Lean and Rocq endpoints; best immediate target for safe refactoring	Current proof has reusable but bulky source, site, and semantic layers.
CM215	Existing proposals include a compiled exact dense/Core transport to `KDense215.machine215`	Uses a three-clause invariant and a conditional entry case; adapter integration must be preserved exactly.
M124, M160	Particularly small marker argument: four changed rows and unconditional entry-marker conditions	Confirm exact table identity and integrate the appropriate adapter and endpoint client. These are existing alternatives.
ZC215	DROP/NIP prefix credit protects an arbitrary raw tail without charging axiom outputs as reserve	Its source scheduler, source equivalence, initialization, and exact binary runtime still need a coherent endpoint.
GC–ZC215	Combines related logical and scheduling savings	More interfaces change; source-level elegance does not by itself reduce formal integration effort.

M124's ordinary transport argument uses predecessor markers distinguishing the two old decoder roles and two short, positive replacement traces. Unlike CM215's invariant, it has no conditional 176/156 entry exception. This makes M124 a good choice for a short explanatory transport proof. M160 has a similarly simple four-row pattern. Existing incoming work already supplies generic marker-family checks; they should be reused rather than reintroduced as new constructions [9, 4].

CM215 currently has the stronger ready-to-use evidence for the precise dense/Core bridge. If the immediate goal is another exact formal endpoint, begin with that compiled adapter. If the goal is to

minimize the local invariant a reader must understand, M124 is the cleaner example. A theorem that invokes the accepted K215 endpoint is deliberately an incremental proof; it inherits that endpoint's logical foundation.

For future changes to the logical language or number of emitted heads, ZC215 offers the most reusable scheduler contract. Its existing codec proves a prefix count inequality. The next section supplies a generic execution-soundness theorem that uses this inequality without depending on the axiom emitter's arity.

6 AX9: move preservation of the old top into the witness word

6.1 The complete arithmetic state

Write the command state as (t, w, d, h): top, paired stack, additive parameter, and cache. Let $\mathcal{R}(d, h; c, x, y)$ be the list of eight logical rows emitted by AX, and let $\mathcal{G}(d, h; c, x, y)$ be its gated bundle. For a finite list L, write $[L; w]$ for right-nested Cantor pairing above a raw natural tail w. The tail need not decode as a well-founded list.

The parent's ten-head command has the interface

$$\mathsf{AX}_{10}(t, w, d, h) = \big(\mathcal{G}, [\mathcal{R}; Q_d(h); t; w], 0, 0\big).$$

The command called DROP in this proof language *pushes* the old top and replaces it by the cache:

$$\mathsf{DROP}(t, w, d, h) = (h, [t; w], d, h).$$

Its name should not be used to infer an ordinary stack-pop convention.

Define a clear-top nine-head emitter by

$$\mathsf{AX}_9(t, w, d, h) = \big(\mathcal{G}, [\mathcal{R}; Q_d(h); w], 0, 0\big).$$

It emits the same eight rows, generalized head, gate, and resets, but does not preserve the incoming top.

Theorem 6.1 (Exact saved-top factorization)**.** *For every natural tuple and every command field choice,*

$$\mathsf{DROP}; \mathsf{AX}_9 = \mathsf{AX}_{10}.$$

Proof. DROP replaces w by $[t; w]$, keeps d, h, and changes the current top to h. AX9 ignores that current top. Substitution gives exactly the parent's full returned tuple. No theoremhood or list-decoding hypothesis is used. □

This is the compiled theorem `ax10_factor`. Replacing every AX10 instruction in an old finite witness program by DROP;AX9 preserves its behavior. The additional DROP costs proof-word length. It need not cost interpreter control states, because DROP is already part of the language.

The observation is a general source-minimization principle: when an emitter always saves a piece of state, consider moving that operation into the compiler for witness programs, provided the enumeration language already supplies the necessary operation.

6.2 The concrete source edit and a cheaper biased version

In the exact source, the closed 19-leaf subtree at path

`11010010010000`

occupies primitive ordinals $[26, 45)$. With zero arithmetic scratch registers it performs

$$(T, W, X, Y) = (t, w, 0, 0) \longmapsto (0, P(t, w), 0, 0).$$

The following seven-leaf block adds D into T while restoring D. The following generalized-head construction uses the resulting T as binder.

Replacing the 19-leaf subtree by a two-leaf clear routine produces the exact clear AX9 interface. It removes 17 primitive occurrences. Deleting the subtree altogether removes 19 occurrences, but changes the binder:

$$\mathsf{AX}_9^+(t, w, d, h) = (\mathcal{G}, [\mathcal{R}; Q_{t+d}(h); w], 0, 0).$$

This biased version is a materially different command semantics. It cannot be justified by silently treating the incoming top as zero.

In particular, the unrestricted DROP factorization is false for AX_9^+: DROP sets the top to h, so the generalized head would be $Q_{h+d}(h)$. The correct identities are

$$\begin{aligned} \mathsf{DROP}; \mathsf{AX}_9^+ &= \mathsf{AX}_{10} && \text{when } h = 0, \\ \mathsf{AX}_9^+(0, [0; w], d, h) &= \mathsf{AX}_{10}(0, w, d, h). \end{aligned}$$

Both conditional identities are proved in `AX9Factor.v`. The conditions are part of the statements.

Source before reassociation	Leaves	Shared subtrees	Depth	Max. zero run
Parent K215	4,751	202	47	6
Clear AX9	4,734	205	47	6
Biased AX9$^+$	4,732	204	47	6

These are the exact raw metrics from the delivered source verifier. The physical allocation evidence is kept separately from these syntactic measurements.

6.3 Repairing the strong compiler for the biased emitter

A strong witness block for theorem code X sends every $(t, w, 0, 0)$ to $(X, w, 0, 0)$, or halts earlier. The initial top and raw tail are arbitrary. This strong contract makes it possible to compose compiler blocks without requiring a decodable or theorem-valued tail.

Write $D, N, M(i), A(c; x, y)$ for DROP, NIP, MP, and AX_9^+. Let

$$\mathsf{CLEAR} = M(1); D; A(1; 0, 0); N^{10}, \qquad J(0) = \varepsilon, \quad J(i) = M(i) \ (i > 0).$$

The first $M(1)$ fails in the parent implication comparison and yields a zero top and cache with parameter one. DROP inserts a zero sentinel. The AX call has a zero gate at coincident parameters

and resets the parameter and cache. Its nine heads plus the sentinel are removed by ten NIPs. A NIP that halts sooner already satisfies the strong-block alternative.

All existing compiler calls to AX10 with cache zero transport through $D; A$ by the restricted factorization. Generalization is the exceptional call: its cache holds the theorem to be generalized. It therefore needs a new sentinel placement.

Given a strong block B_X, use

$$\mathsf{CLEAR}; D; B_X; D; J(i); N; A(i; 0, 0); N^9; D; N^2.$$

Its normal-return checkpoints are

Prefix	State
$\mathsf{CLEAR}; D$	$(0, [0; w], 0, 0)$
then B_X	$(X, [0; w], 0, 0)$
then $D; J(i)$	$(0, [X; 0; w], i, 0)$
then N	$(0, [0; w], i, X)$
then $A(i; 0, 0)$	$(0, [\mathcal{R}; Q_i(X); 0; w], 0, 0)$
then N^9	$(0, [0; w], 0, Q_i(X))$
then $D; N^2$	$(Q_i(X), w, 0, 0)$

The sentinel is part of the raw stack $[0; w]$ preserved by B_X: it lies above the original tail w and below any temporary heads created by B_X. At the exceptional AX entry the top is zero, so the biased binder is exactly i. Earlier halting remains permitted at every NIP. This supplies the written strong-compiler argument for the modified generalization case.

The packet distinguishes this written construction from the compiled identities. It does *not* contain a fully formalized AX9 strong compiler, a complete native-loop equivalence, or a binary runtime/blank-start theorem. Those remain necessary integration work before the new source can certify a smaller consistency machine.

7 An arity-independent soundness theorem for zero-credit schedules

7.1 Why count proofs are only half the interface

The existing ZC215 codec gives the inequality

$$\#\mathsf{DROP}(u) \geq \#\mathsf{NIP}(u)$$

for every prefix u of the concatenated exhaustive schedule $W(0)W(1)W(2)\cdots$. The inequality is not asserted for each candidate word separately. This is a resource statement. A separate semantic theorem is needed to show that execution cannot read an untrusted raw tail or halt because of a value found there.

Fix an arbitrary natural tail J. A protected state has a witness list L such that

$$w = [L; J], \qquad \mathrm{Good}(t), \qquad \mathrm{Good}(h), \qquad \forall a \in L\ \ \mathrm{Good}(a).$$

No condition is placed on J. In particular, J may contain a falsity code, may fail to decode as a formula, or may have the zero alias of Cantor pairing.

Assign the budget changes

$$\mathsf{DROP} : +1, \qquad \mathsf{NIP} : -1, \qquad \mathsf{MP} : 0, \qquad \mathsf{AX} : 0.$$

AX may physically push many theorem-valued heads, but the proof does not count them as credit.

Theorem 7.1 (Protected-tail execution soundness)**.** *Assume* $\mathrm{Good}(0)$*, the pairing projection equations for DROP and NIP, successful MP preservation, theorem-valued fixed AX rows and gate, and* $\mathrm{Good}(h) \Rightarrow \mathrm{Good}(Q_i(h))$ *for every* i*. Let the protected list have length at least an initial credit* b*. If* $b + \#\mathsf{DROP}(u) \geq \#\mathsf{NIP}(u)$ *for every executed prefix* u*, then execution from that protected state cannot expose the opaque tail through NIP. Every successful prefix remains protected with enough explicit heads for its remaining budget. If execution halts by testing the distinguished falsity code 1, then* $\mathrm{Good}(1)$*.*

Induction invariant. The length of the explicit protected list is at least the current credit. DROP adds a theorem-valued head and one unit of budget. A NIP is permitted only at positive budget; hence the list has an explicit head, and the two pairing projection equations identify that head and the remaining paired tail. MP preserves protection without reducing the budget. AX can prepend an arbitrary theorem-valued finite list and resets its scalar outputs as specified; it never needs to supply credited reserve. The halting branch tests a value already known to satisfy Good. Induction on the executed command word proves the result. □

The bridge has three stages: `prefix_counts_credit` establishes the budget, `credit_run_sound` propagates protection, and `zero_credit_semantic_soundness` starts with no explicit protected heads and arbitrary J. The pairing part of the argument uses only

$$\mathrm{hd}(\mathrm{cons}(a, w)) = a, \qquad \mathrm{tl}(\mathrm{cons}(a, w)) = w.$$

These equations remain valid at the zero alias $P(0, 0) = 0$. The protected list is a logical witness, not a unique decoding of the raw natural stack.

This theorem permits safe changes to the number of AX outputs without rewriting stack-reserve arithmetic. It does not authorize deleting logical axioms: completeness is a separate obligation. The emitted rows and gate are abstract theorem-preserving interfaces in this standalone Coq module; their instantiation with the calibrated numerical calculus is inherited work to connect explicitly.

7.2 Clean zero stacks: finite ghost heads remove the credit premise

There is a further simplification when the raw stack is exactly zero. Cantor pairing satisfies $P(0, 0) = 0$. Consequently, for every natural n,

$$\mathrm{prepend}(\underbrace{[0, \ldots, 0]}_{n \text{ heads}}, 0) = 0.$$

If code zero is theorem-valued, the same raw zero stack therefore admits a protected-list witness of any prescribed finite length. This adds no data to the command state or Turing tape; it changes only the witness used in the proof.

Theorem 7.2 (Unconditional command-order soundness from zero stack)**.** *Under the command-preservation hypotheses above, assume* $\mathrm{Good}(0)$ *and* $\mathrm{cons}(0, 0) = 0$*. For every finite command word* u*, theorem-valued initial top and cache, and every initial counter, a halt from* $(t, 0, d, h)$ *implies* $\mathrm{Good}(1)$*. No prefix-credit or command-order assumption is required.*

Proof. Take $n = |u|$ and represent the raw zero stack by n zero-valued ghost heads above tail zero. Every prefix contains at most n NIPs, so its withdrawal count is at most the initial credit n, even without counting any DROP. Apply the protected-tail execution theorem with that initial list and budget. A halting trace has finite length, so this finite witness suffices; no infinite decoded stack or coinduction is necessary. □

The new compiled theorems are `zero_stack_semantic_soundness` and `zero_stack_halt_refutes`. Together with the identities and resource bridge, the delivered 301-line Coq source prints twelve closed assumption reports and passes recursive `coqchk`. Its hypotheses remain explicit parameters, including theorem preservation of the actual row generator and gate.

This clean-zero-stack theorem and the dirty-tail credit theorem solve different problems. Arbitrary tails do not in general equal stacks of harmless ghost zeros, so the prefix-credit condition cannot be dropped there. For an existing command interpreter with a clean zero stack, however, this gives a direct route to separating soundness from enumeration: bind its command-preservation lemmas to this invariant, and leave enumeration only in completeness. The supplied exact command model uses the biased AX9 emitter; applying the idea to the accepted interpreter still requires its concrete command adapter. No new exact-machine endpoint is asserted.

8 State-reduction experiments and their limits

8.1 Context-sensitive merging

A useful fold must preserve arbitrary exterior tape contents under a proved local entry condition. Testing only a blank run or a few small register values is insufficient. The supplied search first proves local bit implications by backward reachability, then checks candidate replacement traces symbolically while treating cells outside the known window as unknown.

The broad K215 experiment examined 202 shared-row state pairs, used 889 proved local-bit facts in a radius-four neighborhood, tested 172,912 candidate replacement rows, and visited 4,965,734 symbolic configurations. Its trace/window limits are 96 steps and 14 cells. A cap exhaustion is recorded as unknown; it is never accepted as success. No valid one-state reduction was found in this template.

An earlier narrower predecessor-marker experiment considered 19 fold pairs, 16,264 provider rows, and 350,893 symbolic configurations. It also considered 81 disjoint two-fold pairs attempting to share one new selector, a net one-state-saving template; no exact two-step provider succeeded. These are complementary bounded searches, not a proof that no 214-state machine exists.

8.2 The source/physical-state tradeoff

The AX9 edits are promising semantically because they remove work that can be inserted into witness programs instead. Their first literal implementations nevertheless have more distinct shared subtrees than the parent. This is a concrete warning against treating primitive count as a proxy for physical states.

Closed reassociation searches did not improve the initial AX9 subtree counts. An open reassociation of the biased version retained 204 shared subtrees while reducing maximum depth from 47 to 39. The allocator's exact prefix/entry identity control reproduces primary K215 byte for byte, including

its eight contextual 108 aliases. Thus a failed price for AX9 is not being inferred from an allocator that also rejects its positive control.

The first feasible protected-row allocation costs were 218 states for clear AX9 and 217 for the biased AX9^{+} source. A more speculative top-binder source prices at 216, but its variable-name completeness calibration is not established. Cheaper budgets were infeasible only within this protected-row allocation model. The delivered records contain the exact controls, bounds, and allocation outputs. These allocations are synthesis/pricing evidence. They do not carry new quantified runtime or blank-start correctness proofs, and none is promoted to an accepted consistency machine. The more radical proposal to use the incoming top itself as the generalized binder also requires a fresh variable-name completeness calibration; it is deliberately not asserted as an equiconsistent candidate here.

8.3 The next concrete synthesis target

The most useful next experiment is to combine an arity-independent zero-credit scheduler with the exact clear-AX9 factorization, then optimize the changed prefix *together with* its consumers and allocation. The biased version is available if its repaired strong compiler is formalized. Reassociation should be judged on complete guarded control flow and entry-context obligations, followed by physical allocation. A reduction in source leaves or proof-graph vertices is evidence for neither an accepted table nor a binary state saving.

A successful 214-state result would still require the full chain: exact row table, deterministic reconstruction, source correspondence, universally quantified primitive runtime, blank startup, and the numerical-calculus consistency calibration. Local positive traces are useful lemmas inside that chain; they do not substitute for it.

9 Integration plan and reproduction

9.1 Immediate integration: the semantic patch

Apply `seed_semantics/seed-semantics.patch` only after verifying the baseline hashes. Rebuild the affected downstream dependency closure and both endpoint clients with the repository's pinned Lean version. Repeat the axiom report. This is the smallest concrete integration task in the packet because it preserves the existing public statements and all machine/program definitions.

The local clean replay is:

```
python3 seed_semantics/replay.py --lean /absolute/path/to/lean
```

The supplied dependency snapshot makes this local replay independent of network access. The final artifact does not include compiled proof objects or compiler binaries.

9.2 Drain and AX9 replay

Use the component drivers documented in `drain_normalization/README.md` and `ax9/README.md`. They compile the new modules and can run the recursive Coq kernel checker. The concrete graph generator is untrusted; its output is checked against literal semantic premises. The source-edit

verifier reconstructs both AX9 sources and compares retained control-flow edges. Integrity manifests check bytes, not semantic correctness.

The full normalizer theorem is conditional only on its stated graph/rank premises. The concrete instance discharges those finite premises. The source-to-graph bridge and a new binary AX9 endpoint are not silently imported as axioms.

9.3 Priorities after integration

First, integrate the checked seed normal form and keep its typed API audit. Second, finish adoption of the already delivered normalized source certificate and exact marker adapters where that integration remains open. Third, connect the zero-credit codec to the new opaque-tail semantic theorem. Fourth, use the drain interface or AX9 source edit where it removes a real implementation mismatch. These are concrete, separable proof steps with reviewable hypotheses.

The new work demonstrates that additional proof simplification is still possible without changing K215's mathematical target. It also supplies a more flexible source-level interface for future state minimization. The state record in this packet remains 215.

A Archive contents and trust boundaries

Path	Purpose
`report.tex`, `report.pdf`	This article and its editable source.
`seed_semantics/`	Exact two-file Lean patch, full 26-module local closure, typed audit, originals, logs, manifests, and replay driver.
`drain_normalization/`	Universal Coq drain/rank proofs, literal graph and rank inputs, generated instance, independent review, successful build/audit records, and generator.
`ax9/`	Universal command/resource Coq theorem, written compiler argument, deterministic source edits, complete edge checks, logs, notices, and baseline source.
`state_search/`	Bounded context-fold searches, allocation controls/prices, and reproducible evidence.
`provenance.json`	Repository commits, input identities, and result scopes.
`MANIFEST.sha256.json`	Archive integrity. No logical claim follows from a hash match alone.
`verify_manifest.py`	Checks all listed archive file hashes.

The seed and AX9 components were developed independently in parallel. The drain theorem was separately reviewed for zero/positive counters, nonzero exit increments, finite-fuel cycles, terminal nodes, and rank-zero behavior. Verification receipts distinguish a fresh local proof build from reuse of the existing K215 theorem. No repository commit, publication, or external message was performed as part of this deliverable.

References

[1] ProveIt, *K215: 215 working states for ZFC consistency*, pinned candidate record. https://github.com/VladimirReshetnikov/ProveIt/blob/56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3/Computability/BusyBeaver/Research/ZFC/candidate-k215/README.md.

[2] ProveIt, *K215 verification record*. https://github.com/VladimirReshetnikov/ProveIt/tree/56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215.

[3] ProveIt, *Ordinary certificates and shared semantic lemmas*, phase five. https://github.com/VladimirReshetnikov/ProveIt/blob/56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase5/README.md.

[4] ProveIt, *Proof-simplification proposals for the K215 endpoints*, proposal index. https://github.com/VladimirReshetnikov/ProveIt/blob/56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/README.md.

[5] ProveIt incoming research, *K215 automatic certificates, concrete credit proofs, and the 214-state search*. https://github.com/VladimirReshetnikov/ProveIt/tree/56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/automatic-certificates.

[6] ProveIt, *K215 Proof and Quotient Detachment*, incoming archive reviewed at the snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3/docs/incoming/K215_Proof_and_Quotient_Detachment_2026-10-21.zip.

[7] ProveIt, *KSeed222 and KSeedModel222*, exact Lean sources. https://github.com/VladimirReshetnikov/ProveIt/blob/b07002a86bcc96e76a31b57dce23d3b8c2b76fa6/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KSeed222.lean. Companion source has the same directory and name `KSeedModel222.lean`.

[8] ProveIt incoming research, *Further simplification: contract 1,059 drain loops*. Inherited graph, counts, and mathematical proposal. https://github.com/VladimirReshetnikov/ProveIt/blob/56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/affine-cores/structural/REPORT.md.

[9] ProveIt, *DR215 marker family*, existing local transport arguments. https://github.com/VladimirReshetnikov/ProveIt/tree/56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3/Computability/BusyBeaver/Research/ZFC/candidate-k215/alternatives/dr215/markers.

[10] ProveIt, *Command-permuted K216 proof*, inherited command equations and compiler. https://github.com/VladimirReshetnikov/ProveIt/blob/56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3/Computability/BusyBeaver/Research/ZFC/candidate-k216/alternatives/permuted/proof.md.

[11] ProveIt, *Zero-credit alternative*, prior tag-lowering and scheduler work. https://github.com/VladimirReshetnikov/ProveIt/tree/56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3/Computability/BusyBeaver/Research/ZFC/candidate-k215/alternatives/zero-credit.